100 Project Interview Questions and Answers

Git Collaborator Access Manager

Deep-dive study guide for Java backend and full-stack interviews
Verified against the repository implementation on branch **testing**
Generated 29 September 2026

How to use this guide

Read the simple answer first. Use the “Interview depth” paragraph when the interviewer asks why, what could fail, or how you would improve it. Source references show where to locate the implementation. Diagrams are deliberately small and explain the relationships most likely to be discussed.

Accuracy note

Answers describe the current code, even when the implementation has a limitation. Recommendations are clearly labeled as improvements. No secrets or environment values are included.

Table of Contents

Table of Contents 2

1. Project Overview and Business Problem 3

Project in one view . . . 3

2. High-Level and Low-Level Architecture 5

Runtime architecture . . . 5

3. Spring Boot, Controllers, and Services 7

Backend layers . . . 7

4. Database, JPA, Prisma, and Transactions 9

Core ER model . . . 9

5. REST APIs and Data Flow 11

Example request flow . . . 11

6. GitHub Integration and Synchronization 13

Sync sequence . . . 13

7. Frontend, Next.js, and UI Flows 15

UI navigation . . . 15

8. Authentication, Authorization, and Security 17

Current authentication boundary . . . 17

9. Docker, Configuration, and Deployment 19

Container topology . . . 19

10. Testing, Error Handling, Performance, and Improvements 21

Improvement roadmap . . . 21

Rapid Revision Sheet 23

A strong closing answer . . . 23

1. Project Overview and Business Problem

Project in one view

```
GitHub repositories + collaborators
              | sync
              v
       Local access inventory
              | map identities
              v
     Employees <-> GitHub users
              | offboard / approve / audit
              v
       Controlled access removal
```

How to explain the diagram: Start with GitHub data, then show how identity mapping enables governed removal.

1. What problem does this project solve?

Simple answer: It gives an administrator one place to see GitHub repository access, connect GitHub accounts to employees, and remove access when someone leaves.

Interview depth: The real value is governance: GitHub identity data is converted into employee-centered workflows, job records, approvals, and audit history.

Verified source: *README.md; DashboardService; OffboardingService*

2. Who is the main user?

Simple answer: The implemented user is a single administrator configured through environment variables.

Interview depth: The token type mentions ADMIN and VIEWER, but login always creates ADMIN and no role rule is enforced.

Verified source: *AuthController.java; lib/auth.ts*

3. What are the main features?

Simple answer: Sync GitHub data, browse repositories and collaborators, manage employees, map identities, remove access, approve bulk offboarding, and inspect audit/sync history.

Interview depth: Each feature has a UI flow, a Next.js API boundary, a Spring service, and PostgreSQL records.

Verified source: *src/app; backend/controller; prisma/schema.prisma*

4. How would you explain the project in 30 seconds?

Simple answer: It is a GitHub access-management application built with Next.js, Spring Boot, and PostgreSQL. It imports repository access, maps GitHub users to employees, and automates audited offboarding.

Interview depth: Mention the external GitHub API, the layered backend, and the approval flow to show full-stack understanding.

Verified source: *Repository-wide verified architecture*

5. Why is employee mapping necessary?

Simple answer: A GitHub username is a technical identity. Mapping it to an employee lets the company ask who owns the access and remove all access for that person.

Interview depth: The GithubUser table has an optional employee_id, so unmapped users can still exist after sync.

Verified source: *Employee.java; GithubUser.java; schema.prisma*

6. What is the difference between sync and offboarding?

Simple answer: Sync reads GitHub and builds the local inventory. Offboarding changes GitHub access and then updates the local inventory.

Interview depth: Sync is mainly reconciliation; offboarding is a destructive command workflow with partial-failure tracking.

Verified source: *SyncService.java; OffboardingService.java*

7. What does governance mean in this project?

Simple answer: It means recording approvals and audit events around sensitive access changes.

Interview depth: The strongest implemented governance case is bulk offboarding: a batch waits for an ApprovalRequest before execution.

Verified source: *GovernanceService.java*

8. What is implemented versus only documented?

Simple answer: Implementation means executable code exists. Some older docs mention hosting ideas, but current code verifies Docker and Render configuration.

> **Interview depth:** Always answer from controllers, services, schemas, and deployment files before trusting design documents.

Verified source: *docker-compose.yml; render.yaml; git-collabration/README.md*

9. What is the most important end-to-end use case?

Simple answer: Sync GitHub, create an employee, map a GitHub user, then offboard that employee and remove repository access.

> **Interview depth:** The integration test exercises exactly this lifecycle with a mocked GitHub client.

Verified source: *OnboardingWorkflowIntegrationTest.java*

10. What makes this project suitable for a Java interview?

Simple answer: It demonstrates REST controllers, service layering, JPA relationships, transactions, external APIs, error handling, and Docker.

> **Interview depth:** It also gives honest improvement topics: Spring Security, DTO validation, migrations, and scalability.

Verified source: *backend/pom.xml and Java packages*

2. High-Level and Low-Level Architecture

Runtime architecture

```
Browser
  | HTTP pages + /api + signed cookie
  v
Next.js (UI + API gateway)
  | HTTP JSON via SPRING_BOOT_URL
  v
Spring Boot (controllers -> services -> repositories)
  | JPA/JDBC                    | HTTPS bearer PAT
  v                              v
PostgreSQL                  GitHub REST API
```

How to explain the diagram: Follow one request downward from browser to data/external API, then return the response upward.

11. What is the high-level architecture?

Simple answer: The browser talks to Next.js. Next.js calls Spring Boot. Spring Boot reads PostgreSQL and calls GitHub.

Interview depth: Nginx can sit in front in the production Compose overlay. Next.js also has limited direct Prisma access.

Verified source: *docker-compose.yml; nginx.conf; lib/backend.ts*

12. Why are there both Next.js API routes and Spring controllers?

Simple answer: Next routes give the browser a same-origin API and act as a server-side gateway. Spring owns the main business rules.

Interview depth: This resembles a backend-for-frontend pattern, although some logic is duplicated and the boundary is not perfectly clean.

Verified source: *src/app/api; backend/controller*

13. What is the request path for a normal button click?

Simple answer: Client component -> /api route in Next -> TypeScript service -> Spring controller -> Spring service -> repository or GitHub -> response -> UI refresh.

Interview depth: This path keeps database credentials and the GitHub PAT away from the browser.

Verified source: *components; services; lib/backend.ts*

14. How is the backend layered?

Simple answer: Controllers handle HTTP, services implement workflows, repositories perform persistence, and entities map tables.

Interview depth: There is no DTO layer; controllers use maps and expose entity-shaped responses.

Verified source: *backend/controller, service, repository, model*

15. What is a backend-for-frontend?

Simple answer: It is a server layer designed for one frontend. Here, Next route handlers translate browser requests into Spring calls.

Interview depth: The BFF can centralize cookies, response shaping, and same-origin access, but it should also consistently enforce authentication.

Verified source: *src/app/api; middleware.ts*

16. Why is business logic mainly in services?

Simple answer: It keeps controllers small and makes workflows reusable and transactional.

Interview depth: OffboardingService is large because it coordinates many related workflows; splitting command handlers could improve maintainability.

Verified source: *OffboardingService.java*

17. What is duplicated across layers?

Simple answer: Authentication endpoints and GitHub clients exist in both Next and Spring. Repository-removal permission checks are also repeated.

Interview depth: Duplication can drift and causes extra GitHub calls; choose one policy owner.

Verified source: *GitHubClientService.java; services/github/client.ts*

18. How do components depend on each other?

Simple answer: Controllers depend on services; services depend on repositories and integration clients; entities define relationships.

Interview depth: Spring dependency injection wires fields with @Autowired, including a @Lazy reference used to break the GovernanceService/OffboardingService cycle.

Verified source: *GovernanceService.java; OffboardingService.java*

19. Is this a microservice architecture?

Simple answer: No. It is a small distributed web application with a frontend server and one Spring backend service.

Interview depth: Calling it microservices would be misleading because the Java business domain is one deployable service and one database.

Verified source: *docker-compose.yml*

20. What architectural improvement would you make first?

Simple answer: Enforce authentication at both the Next API and Spring API boundaries.

Interview depth: After security, choose one migration owner and remove duplicated GitHub/policy logic.

Verified source: *middleware.ts; Spring controllers; schema configuration*

3. Spring Boot, Controllers, and Services

Backend layers

```
HTTP request
   v
@RestController  - status, path, basic validation
   v
@Service         - business workflow + transactions
   v
JpaRepository    - generated queries / JPQL
   v
PostgreSQL
```

How to explain the diagram: Explain that each layer has one main responsibility and depends on the layer below.

21. Why is Spring Boot used?

Simple answer: It provides an embedded web server, REST support, dependency injection, validation libraries, JPA, and test utilities.

Interview depth: The project uses Spring Boot 3.4.1 with Java 23.

Verified source: *backend/pom.xml*

22. What does @RestController do?

Simple answer: It marks a class as an HTTP controller whose return values are serialized as JSON responses.

Interview depth: Every backend controller uses it, and ResponseEntity controls status codes and headers.

Verified source: *backend/controller/*.java*

23. What does @RequestMapping do?

Simple answer: It sets a shared URL prefix for a controller, such as /api/repositories.

Interview depth: Method annotations then add GET, POST, PATCH, or DELETE behavior.

Verified source: *RepositoryController.java*

24. Why use ResponseEntity?

Simple answer: It lets the controller return a body, HTTP status, and headers explicitly.

Interview depth: Examples include 201 for creation, 302 for login redirects, 403 for missing GitHub admin permission, and 404 for missing resources.

Verified source: *backend/controller*

25. How does dependency injection work here?

Simple answer: Spring creates services and repositories and injects them into fields marked @Autowired.

Interview depth: Constructor injection would make dependencies clearer and easier to test, but field injection is the current implementation.

Verified source: *backend Java classes*

26. What does @Transactional provide?

Simple answer: It groups database operations so they commit together or roll back on failure.

Interview depth: EmployeeService, GovernanceService, and OffboardingService are transactional at class level.

Verified source: *service/*.java*

27. Why use REQUIRES_NEW?

Simple answer: It starts an independent transaction for one repository sync or one access deletion.

Interview depth: A failure in one item does not force all earlier successful items to roll back, enabling PARTIAL_SUCCESS.

Verified source: *SyncTransactionService; OffboardingTransactionService*

28. How is a circular service dependency handled?

Simple answer: GovernanceService injects OffboardingService with @Lazy.

Interview depth: The cycle exists because approval triggers bulk execution, while batch creation asks GovernanceService to create an approval. A separate workflow orchestrator would be cleaner.

Verified source: *GovernanceService.java; OffboardingService.java*

29. How are controller inputs validated?

Simple answer: Controllers manually check required fields and parse UUID values inside try/catch blocks.

> **Interview depth:** spring-boot-starter-validation is installed, but DTO classes with @Valid are not implemented.

Verified source: *pom.xml; controllers*

30. How are exceptions handled?

Simple answer: Most controller methods catch Exception and return HTTP 400 with an error message.

> **Interview depth:** There is no @RestControllerAdvice, typed exception hierarchy, or consistent error code, so client behavior is less precise.

Verified source: *backend/controller*

4. Database, JPA, Prisma, and Transactions

Core ER model

```
Employee 1 ---- * GithubUser
GithubUser 1 ---- * RepositoryAccess * ---- 1 Repository
Employee 1 ---- * OffboardingJob 1 ---- * JobItem
GithubUser 1 ---- * RemovalRequest * ---- 1 Repository
ApprovalRequest 1 ---- 0..1 BulkBatch 1 ---- * BatchItem
```

How to explain the diagram: RepositoryAccess is the central many-to-many bridge; workflow tables preserve operational history.

31. Which database is used?

Simple answer: PostgreSQL 16 is used in Docker, with JDBC for Spring and DATABASE_URL for Prisma.

Interview depth: The database runs on container port 5432 and defaults to host port 5433.

Verified source: *docker-compose.yml; application.properties*

32. What is JPA?

Simple answer: JPA is the Java standard for mapping objects to relational tables. Hibernate is the runtime implementation used by Spring Data JPA.

Interview depth: Entities use @Entity, @Table, @Id, relationships, lifecycle callbacks, and column mappings.

Verified source: *backend/model*

33. What is Prisma doing in a Spring project?

Simple answer: Prisma defines the same PostgreSQL schema, generates TypeScript types/client code, and supports a few direct Next operations.

Interview depth: Two ORMs against one schema create a drift risk and should have one migration owner.

Verified source: *prisma/schema.prisma; lib/prisma.ts*

34. What are the central entities?

Simple answer: Employee, GithubUser, Repository, and RepositoryAccess form the core access model.

Interview depth: Workflow entities add SyncJob, RemovalRequest, OffboardingJob, ApprovalRequest, BulkOffboardingBatch, and audit records.

Verified source: *backend/model; schema.prisma*

35. How is the many-to-many access relation modeled?

Simple answer: RepositoryAccess connects one GitHub user to one repository and stores permission, source, and sync time.

Interview depth: A unique constraint on repository_id plus github_user_id prevents duplicate access rows.

Verified source: *RepositoryAccess.java; schema.prisma*

36. What happens when a collaborator is removed locally?

Simple answer: The matching RepositoryAccess row is deleted. If the GitHub user has no remaining access, its employee mapping is cleared.

Interview depth: This behavior is implemented inside an isolated transaction.

Verified source: *OffboardingTransactionService.java*

37. How are timestamps maintained?

Simple answer: Entities set timestamps in field defaults and @PrePersist/@PreUpdate methods; Prisma also defines now() and @updatedAt.

Interview depth: Because both ORMs can write, timestamp behavior should be tested for consistency.

Verified source: *model classes; schema.prisma*

38. Are database migrations implemented?

Simple answer: No committed migration system is present. Spring uses Hibernate update and Next runs Prisma db push.

Interview depth: For production, use Flyway/Liquibase or Prisma migrations as the single versioned source of truth.

Verified source: *application.properties; docker-entrypoint.sh*

39. Which important constraints exist?

Simple answer: GitHub username and repository GitHub ID are unique; repository access pairs are unique; several foreign keys use cascade or set-null behavior.

> **Interview depth:** Employee email uniqueness is checked by service code, not declared unique in the schema.

Verified source: *schema.prisma; EmployeeService.java*

40. Why can an external API and database not be one transaction?

Simple answer: PostgreSQL can roll back local changes, but it cannot undo a GitHub REST call automatically.

> **Interview depth:** The code calls GitHub first and then updates the database; a DB failure afterward can leave temporary inconsistency requiring reconciliation.

Verified source: *OffboardingService.java*

5. REST APIs and Data Flow

Example request flow

```
EmployeeForm
  -> Next POST /api/employees
  -> employee-service.ts / fetchBackend
  -> Spring EmployeeController.createEmployee
  -> EmployeeService.createEmployee
  -> EmployeeRepository.save
  -> PostgreSQL
  <- 201 { employee } -> router.refresh()
```

How to explain the diagram: Name each boundary so the interviewer sees that you can trace a request end to end.

41. What is the API style?

Simple answer: It uses JSON REST-like resources plus action endpoints such as /execute, /remove-all, and /remove-collaborators.

Interview depth: The action URLs represent business commands that do more than simple CRUD.

Verified source: *controllers; src/app/api*

42. What are the main repository endpoints?

Simple answer: GET repositories, metrics, detail, and access; POST remove-collaborators.

Interview depth: The detail identifier accepts either a UUID or a numeric GitHub repository ID.

Verified source: *RepositoryController.java*

43. What are the main employee endpoints?

Simple answer: List, create, get, update, delete, and map a GitHub user.

Interview depth: Create returns 201, get may return 404, and manual validation errors normally return 400.

Verified source: *EmployeeController.java*

44. How does the sync endpoint behave?

Simple answer: POST /api/sync starts synchronous synchronization and GET returns the latest twenty jobs.

Interview depth: If initial GitHub fetching fails, runSync returns a failure object that the controller still sends as HTTP 200.

Verified source: *SyncController.java; SyncService.java*

45. What is returned by removal operations?

Simple answer: A summary with removed/failed counts and per-repository or per-user results.

Interview depth: This supports partial success and lets the UI explain exactly what failed.

Verified source: *OffboardingService.java*

46. How are query parameters used?

Simple answer: q filters employees, collaborators, and repositories; visibility filters repositories.

Interview depth: Repository queries use case-insensitive JPQL LIKE expressions.

Verified source: *repositories; RepositoryRepository.java*

47. Why are action responses wrapped in maps?

Simple answer: Maps quickly create JSON objects such as {employee: ...} or {result: ...}.

Interview depth: They are flexible but not compile-time safe; response DTOs would improve documentation and stability.

Verified source: *backend/controller*

48. How are status codes selected?

Simple answer: 200 for successful reads/actions, 201 for created records, 302 for auth redirects, 400 for most errors, 403 for permission denial, and 404 for missing detail resources.

Interview depth: Many distinct server problems become 400 because of broad exception catches.

Verified source: *backend/controller*

49. Is API documentation generated?

Simple answer: No OpenAPI or Swagger configuration exists.

> **Interview depth:** The production improvement is springdoc-openapi plus typed DTO schemas and examples.

Verified source: *pom.xml; repository search*

50. How would you trace POST /api/employees?

Simple answer: Browser form -> Next route -> employee-service.ts -> Spring EmployeeController -> EmployeeService -> EmployeeRepository -> PostgreSQL -> JSON response -> UI refresh.

> **Interview depth:** The Next handler also records an audit event after the Spring create call.

Verified source: *employee-form.tsx; api/employees; EmployeeService.java*

6. GitHub Integration and Synchronization

Sync sequence

```
Admin -> POST /api/sync -> create RUNNING SyncJob
  -> GitHub GET /user/repos (paged)
     -> each repo: collaborators (paged)
        -> each collaborator: permission
        -> REQUIRES_NEW upsert repo/users/access
  -> mark missing repositories inactive
  -> finalize SUCCESS / PARTIAL_SUCCESS / FAILED + audit
```

How to explain the diagram: Emphasize pagination, per-collaborator permission calls, isolated repository transactions, and final status.

51. How does GitHub authentication work?

Simple answer: The server reads GITHUB_PAT and sends it as an Authorization: Bearer header.

Interview depth: The token never needs to reach browser JavaScript. Missing token causes an explicit error.

Verified source: *GitHubClientService.java*

52. Which GitHub endpoints are called?

Simple answer: User repositories, repository collaborators, collaborator permission, repository detail, authenticated user, and collaborator deletion.

Interview depth: The code sets the GitHub API version header to 2022-11-28.

Verified source: *GitHubClientService.java*

53. How are all repositories fetched?

Simple answer: The client calls /user/repos with 100 items per page until a page contains fewer than 100.

Interview depth: It sorts by update time descending but does not use GITHUB_OWNER to filter repositories.

Verified source: *GitHubClientService.getRepositories*

54. How are collaborator permissions fetched?

Simple answer: After listing collaborators for a repository, the backend calls the permission endpoint once for every collaborator.

Interview depth: This is accurate but expensive and can approach rate limits for large installations.

Verified source: *SyncService.java*

55. How does rate-limit handling work?

Simple answer: For a 403 with x-ratelimit-remaining equal to zero, it waits until the reset time and retries once.

Interview depth: There is no exponential backoff, jitter, circuit breaker, or handling for secondary rate limits.

Verified source: *GitHubClientService.requestJson*

56. What timeout behavior exists?

Simple answer: Java HttpClient has a ten-second connection timeout.

Interview depth: No per-request response timeout is configured, so a connected request could wait longer than intended.

Verified source: *GitHubClientService.java*

57. How does sync update local access?

Simple answer: For each repository, it upserts the repository, deletes its old access rows, upserts GitHub users, and inserts the current access rows.

Interview depth: The unit runs in REQUIRES_NEW, so one bad repository becomes a failed detail rather than aborting the full sync.

Verified source: *SyncTransactionService.java*

58. How are deleted repositories handled?

Simple answer: After sync, local repositories not present in the fetched GitHub IDs are marked inactive.

Interview depth: Records are preserved for history instead of being deleted.

Verified source: *SyncService.runSync*

59. How is collaborator deletion performed?

Simple answer: The backend sends DELETE /repos/{owner}/{repo}/collaborators/{username}.

> **Interview depth:** A GitHub not-found error is treated as already removed, but detection relies on the exception message containing “not found.”

Verified source: *OffboardingService.java*

60. Why are there two GitHub clients?

Simple answer: Spring has the main client, while Next has another client used in the repository-removal pre-check.

> **Interview depth:** This repeats permission calls and logic. The cleaner design is for Spring to own integration and policy.

Verified source: *Java and TypeScript GitHub clients*

7. Frontend, Next.js, and UI Flows

UI navigation

```
Login -> Dashboard -> Sync Status
          |-> Repositories -> Detail -> Remove selected
          |-> Collaborators -> Remove from all repos
          |-> Employees -> Detail / map / remove access
          |-> Offboarding -> direct or bulk request
          +-> Governance -> approve/reject + audit
```

How to explain the diagram: Show the dashboard as the hub and describe the major destructive flows.

61. Which frontend framework is used?

Simple answer: Next.js 15 App Router with React 19 and TypeScript.

Interview depth: Pages mix server components for data loading with client components for interactive forms and buttons.

Verified source: *git-collabration/package.json; src/app*

62. What is a server component here?

Simple answer: An async page component that loads data on the server and returns HTML.

Interview depth: Repository, employee, dashboard, governance, and status pages fetch through service functions before rendering.

Verified source: *src/app/**/page.tsx*

63. What is a client component?

Simple answer: A component marked use client that uses state, effects, event handlers, and browser fetch.

Interview depth: Examples include SyncButton, EmployeeForm, GovernanceConsole, and removal consoles.

Verified source: *src/components*

64. How is routing handled?

Simple answer: Folders under src/app define routes, and [id] folders define dynamic parameters.

Interview depth: Navigation uses Link; client actions use useRouter for replace, refresh, or navigation.

Verified source: *src/app; components*

65. How is frontend state managed?

Simple answer: Local React useState/useMemo/useEffect handles form, selection, loading, status, and filter state.

Interview depth: There is no Redux, Zustand, or global server-state library.

Verified source: *src/components*

66. How does the UI report progress?

Simple answer: Buttons set loading flags, disable repeated actions, and show inline messages or Toast components.

Interview depth: After success, router.refresh re-renders server data.

Verified source: *SyncButton; removal components*

67. How are destructive actions confirmed?

Simple answer: DialogProvider exposes promise-based confirm and alert dialogs.

Interview depth: Repository and collaborator removal buttons wait for confirmation before making requests.

Verified source: *dialog-provider.tsx*

68. How does repository filtering work?

Simple answer: RepositoryBrowser filters the already-loaded list in the browser by query and visibility.

Interview depth: The backend also supports q and visibility, but this UI component performs its displayed filtering locally.

Verified source: *repository-browser.tsx*

69. What happens when backend reads fail?

Simple answer: Several service functions catch errors and return empty arrays or zero-value objects.

Interview depth: This avoids crashing the page but can make an outage look like “no data,” so visible error states would be better.

Verified source: *frontend services*

70. What is the purpose of fetchBackend?

Simple answer: It builds the Spring URL, adds JSON headers, disables caching, parses errors, and returns typed JSON.

> **Interview depth:** It is the common server-to-server adapter from Next to Spring.

Verified source: *lib/backend.ts*

8. Authentication, Authorization, and Security

Current authentication boundary

```
Request -> Next middleware
  no cookie -> /login
  any cookie -> allowed
      page -> PageShell verifies HMAC
      API route -> no consistent verification
Spring API -> no Spring Security filter

Verified gap: presence is not the same as authentication.
```

How to explain the diagram: The diagram exposes the exact gap: cookie presence is checked earlier than signature verification, and Spring is unprotected.

71. How does login work?

Simple answer: Configured username/password are compared, then an HMAC-SHA256 signed payload is stored in gcam_session.

Interview depth: Next sets HttpOnly, SameSite=Lax, and Secure in production. The Spring version sets HttpOnly but Secure false.

Verified source: *api/auth/login; AuthController.java*

72. What is inside the session token?

Simple answer: A Base64URL JSON payload with username and role, followed by an HMAC signature.

Interview depth: It is not encrypted, so the payload can be read; the signature prevents undetected modification.

Verified source: *lib/auth.ts; SessionManager.java*

73. Does the token expire?

Simple answer: No expiration field is included in the signed payload.

Interview depth: The Next cookie has no explicit maxAge, while the Spring cookie uses 24 hours. A robust token should include expiry and rotation/revocation rules.

Verified source: *auth implementations*

74. How does middleware protect routes?

Simple answer: It redirects to login when the cookie is missing.

Interview depth: It checks only cookie presence, not the HMAC signature. PageShell verifies pages, but API route handlers do not consistently verify a session.

Verified source: *middleware.ts; PageShell*

75. Are Spring endpoints authenticated?

Simple answer: No Spring Security filter or interceptor protects the controllers.

Interview depth: If the backend URL is reachable, callers can directly invoke endpoints. This is the highest-priority production gap.

Verified source: *pom.xml; controllers*

76. Is role-based access control implemented?

Simple answer: No. Role values exist, but actions do not check ADMIN versus VIEWER.

Interview depth: A real design would map permissions to endpoints/actions and enforce them in both layers.

Verified source: *lib/auth.ts; controller code*

77. How are secrets managed?

Simple answer: Values come from environment variables and Render environment groups.

Interview depth: Defaults such as admin/change-me and a placeholder session secret are unsafe if not replaced. Secrets are not printed in this guide.

Verified source: *env.ts; compose; render.yaml*

78. What CSRF protection exists?

Simple answer: No explicit CSRF token mechanism is implemented.

Interview depth: SameSite=Lax helps some cases, but state-changing cookie-authenticated routes should have deliberate CSRF controls.

Verified source: *auth routes; middleware*

79. What security checks exist before repository removal?

Simple answer: The code verifies the PAT has repository admin permission and blocks removing the authenticated admin account.

> **Interview depth:** The check is performed in both Next and Spring, which is safer than UI-only enforcement but duplicates calls.

Verified source: *remove-collaborators routes/controllers*

80. What security improvements should be prioritized?

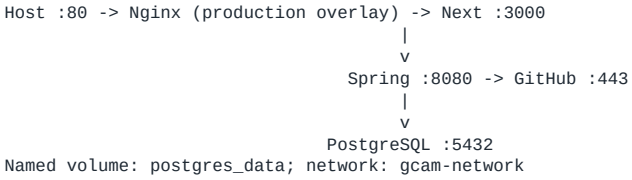
Simple answer: Add Spring Security, verify sessions on every Next API route, add expiry and CSRF protection, reject weak secrets, validate DTOs, and add rate limiting.

> **Interview depth:** Also minimize raw external error details and add CSP/HSTS where TLS termination is controlled.

Verified source: *Verified gap analysis*

9. Docker, Configuration, and Deployment

Container topology



> How to explain the diagram: Separate host ports from internal container ports and show that only Spring calls GitHub.

81. Which containers run locally?

Simple answer: PostgreSQL, Spring Boot backend, and Next.js frontend. The production overlay adds Nginx.

> **Interview depth:** They share the gcam-network bridge and have health checks and log rotation.

Verified source: *docker-compose.yml; docker-compose.prod.yml*

82. How does service startup order work?

Simple answer: Backend waits for a healthy database; frontend waits for healthy database and backend.

> **Interview depth:** Compose dependency health reduces startup races but does not replace application retry logic.

Verified source: *docker-compose.yml*

83. How is database data persisted?

Simple answer: A named Docker volume stores /var/lib/postgresql/data.

> **Interview depth:** Removing containers keeps data; removing the volume deletes it.

Verified source: *docker-compose.yml*

84. How are the images built?

Simple answer: Both use multi-stage builds. Maven builds the JAR; Node installs dependencies and builds Next.

> **Interview depth:** Runtime images contain only the needed artifacts and run as non-root users.

Verified source: *backend/Dockerfile; frontend/Dockerfile*

85. What does the frontend entrypoint do?

Simple answer: It retries prisma db push up to fifteen times, then starts Next.

> **Interview depth:** This helps cold starts but is not a versioned migration process.

Verified source: *docker-entrypoint.sh*

86. What does Nginx provide?

Simple answer: Reverse proxying, gzip, static asset caching, request size limit, timeouts, and some security headers.

> **Interview depth:** It routes all application traffic to Next, not directly to Spring.

Verified source: *nginx/nginx.conf*

87. How is Render configured?

Simple answer: render.yaml defines managed PostgreSQL plus Docker backend and frontend services with shared environment variables.

> **Interview depth:** The frontend calls a configured backend URL and both expose health endpoints.

Verified source: *render.yaml*

88. Are CI/CD pipelines present?

Simple answer: No GitHub Actions or other CI configuration was found.

> **Interview depth:** Render may deploy from repository changes externally, but that automation is not defined as a checked-in workflow.

Verified source: *repository file inventory*

89. Why does the backend Docker build skip tests?

Simple answer: The image build uses mvn clean package -DskipTests to reduce build dependency/time.

> **Interview depth:** A separate CI test stage should run tests before image publication.

Verified source: *backend/Dockerfile*

90. What configuration risk exists across environments?

Simple answer: Different defaults, published backend/database ports, and stale README hosting notes can confuse deployment expectations.

> **Interview depth:** Keep one deployment runbook and validate required production variables at startup.

Verified source: *compose; render; READMEs*

10. Testing, Error Handling, Performance, and Improvements

Improvement roadmap

```
1. Enforce auth + authorization
2. Choose one versioned migration system
3. DTO validation + global error contract
4. Remove duplicated GitHub logic
5. Targeted queries + async jobs + rate control
6. Frontend, security, contract, and load tests
```

How to explain the diagram: Present improvements in risk order rather than starting with cosmetic refactoring.

91. What automated tests exist?

Simple answer: A Spring integration test uses MockMvc and a mocked GitHub client.

Interview depth: It covers health, full sync-to-offboarding, bulk approval/execution, and repository permission validation.

Verified source: *OnboardingWorkflowIntegrationTest.java*

92. What does the integration test verify after sync?

Simple answer: Repository, GitHub user, and access records exist with the expected values.

Interview depth: It then creates an employee, maps the user, creates and executes offboarding, and checks access deletion.

Verified source: *OnboardingWorkflowIntegrationTest.java*

93. What test gaps remain?

Simple answer: No frontend tests, security tests, live GitHub contract tests, load tests, or migration tests were found.

Interview depth: Add component tests, API authorization tests, Testcontainers PostgreSQL, and contract tests around GitHub responses.

Verified source: *repository test inventory*

94. How are errors logged?

Simple answer: Spring enables package DEBUG and SQL display; code uses System.out/err. Next uses console.error. Docker rotates JSON logs.

Interview depth: Structured logging, correlation IDs, and secret redaction are not implemented.

Verified source: *application.properties; services; compose*

95. How are partial failures represented?

Simple answer: Sync and offboarding count errors and set SUCCESS, FAILED, or PARTIAL_SUCCESS-like states.

Interview depth: Per-repository job items and result arrays retain detailed outcomes.

Verified source: *SyncService; OffboardingService*

96. What are likely performance bottlenecks?

Simple answer: GitHub permission calls per collaborator, relationship loading, findAll filtering of removal requests, and synchronous bulk execution.

Interview depth: Large installations need pagination, targeted queries, batching, queues, and controlled concurrency.

Verified source: *SyncService; OffboardingService*

97. What is an N+1 problem in this project?

Simple answer: One initial collaborator list is followed by one permission request per collaborator; ORM relationship traversal can also issue many queries.

Interview depth: Measure both external call count and SQL query count before optimizing.

Verified source: *SyncService; entity fetch strategies*

98. How would you make sync asynchronous?

Simple answer: POST would create a queued job and return 202. A worker would process repositories and update progress.

Interview depth: The UI could poll or use server-sent events, and retries could happen per repository.

Verified source: *Recommended design based on SyncJob model*

99. How would you improve error handling?

Simple answer: Use typed exceptions, @RestControllerAdvice, stable error codes, correct status codes, and visible frontend failure states.

> **Interview depth:** Separate validation, not-found, conflict, authorization, external dependency, and internal errors.

Verified source: *Current controller behavior*

100. What is your final technical assessment?

Simple answer: The project has strong end-to-end workflows and good interview coverage, but needs security enforcement and migration discipline before production use.

> **Interview depth:** Next priorities are auth, one schema owner, DTO/error contracts, deduplicated integration logic, targeted queries, and broader tests.

Verified source: *Repository-wide verified assessment*

Rapid Revision Sheet

Use these points in the last ten minutes before an interview.

• Stack: Next.js 15 + React 19 + Spring Boot 3.4.1 + Java 23 + PostgreSQL 16.

• Core model: Employee -> GithubUser -> RepositoryAccess -> Repository.

• Sync: paged GitHub reads, per-user permission calls, isolated repository transactions, job/audit records.

• Offboarding: GitHub deletion first, local deletion second, partial-failure reporting.

• Governance: bulk batch -> approval -> synchronous per-employee jobs.

• Security truth: signed cookies exist, but Next API and Spring enforcement are incomplete.

• Schema truth: Prisma db push and Hibernate update both act on the database; no migrations.

• Deployment: Compose has DB/backend/frontend; optional Nginx; Render blueprint exists.

• Best improvement answer: security first, migrations second, DTO/errors third, then scalability/tests.

A strong closing answer

> “The project proves the complete business workflow and demonstrates Spring layering, JPA relationships, transactions, external API integration, and full-stack request tracing. I can also explain its current production risks honestly: authentication enforcement, schema ownership, duplicated GitHub logic, synchronous scaling, and test coverage. My improvement plan starts with security and data safety before performance tuning.”